The Agentic Development Handbook

Personal field notes on building software with agents

Abhinav Ragidi

Started June 2026

Contents

Claude Code Basics

> **What this chapter covers:** the foundational mental model of how Claude Code actually works — what happens between the prompt you type and the response you get back. Everything in the next chapter sits on top of this. Blockquotes are exact lines from the source chat.

Source: a chat working through Claude Code internals — the agent loop, brain-vs-hands, the tool menu, model-vs-SDK. Learned June 2026.

The core loop: prompt in, response out

At the surface it's simple — you give Claude Code a **prompt** and you get a **response**. But between those two moments, a lot happens on its own.

> you are involved exactly twice — once when you send the prompt, once when you read the result. Everything in between, Claude runs a loop by itself, and your code is never called during it.

Two parts work together to turn your prompt into that response: the **Claude model** and the **SDK** (the core of Claude Code).

Brain and hands

The cleanest way to hold the split:

> The model is the brain. It reads the context and decides. Its only output is tokens — text, plus structured requests like "call the Bash tool with npm test." It cannot run anything itself: no shell, no file access, no network. It only says what it wants done.
>
> The SDK is the hands. It's ordinary software running on a real machine … that takes the model's request, actually executes it, captures the output, and feeds it back as the next message. It also drives the loop, manages the context window, and enforces permissions.

So the loop is: **prompt → the brain (model) decides + the hands (SDK) act → response.** The brain never touches your machine; the hands never decide *what* to do — they execute what the brain asks.

What "turn by turn" means — the agent loop

You are not in this loop. "Turn by turn" refers to the laps of the loop Claude runs by itself, not human back-and-forth.

> After your prompt, Claude generates a response that can contain text (its reasoning) and/or tool calls … The SDK then actually runs those tools and feeds the output back to Claude, which generates again. Each full cycle — Claude emits tool calls, the SDK runs them, results feed back — is one turn … The loop ends when Claude produces a response with no tool calls, i.e. "I'm done."

The messages inside the loop:

- An **assistant message** is what Claude (the brain) produced on that turn — its text plus any tool-call requests. One is emitted per turn, including the final text-only one.
- A **user message**, *inside the loop*, is not you — it's the tool result handed back:

> the "user" message is not you — it's the tool result being handed back to Claude. The Messages API represents tool output as a user-role message … You authored only the very first one.

Worked example: *"fix the failing tests"* → turn 1, Claude runs the tests → output comes back → turn 2, reads the files → turn 3, edits and re-runs → final turn, text-only "fixed it" → result.

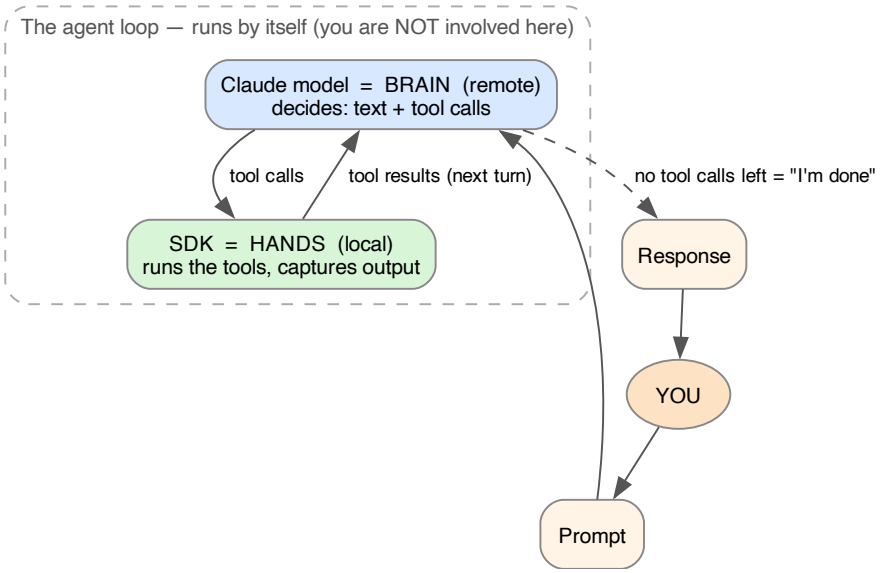

Figure 1: The agent loop. You touch it twice — the prompt in, the response out. In between, the model (brain) and SDK (hands) cycle turn by turn until the model emits no more tool calls.

The tool menu — how the hands know what they can do

The model can't ask for just anything:

> At the start, the SDK hands the model the prompt along with the tool definitions — a menu of tools, each with a name, description, and input schema — and the model can only request items from that menu. Every item has executor code behind it: the SDK ships with built-ins like Read, Write, Edit, Bash, Glob, Grep, and WebSearch … So capability

is guaranteed by construction — the model picks from a list, and everything on the list is runnable.

And **Bash is the universal escape hatch:**

> anything you can type into a shell on that pod — python train.py, gsutil/gcloud, pip install, nvidia-smi — the model gets done by asking Bash to run it.

So "does the SDK have enough capability to run whatever the model asks?" — yes, by construction. What actually bounds capability is three things: (1) which tools you registered, (2) what's installed and reachable on the machine, and (3) what your permissions allow.

Where the brain actually lives (the key correction)

A tempting but wrong picture is "Claude Code SDK = model + harness bundled together." The fix:

> The model is not inside the SDK. The SDK is a standalone package that runs the agent loop … and it runs locally, on your machine. The model runs on Anthropic's servers. Every turn, the SDK packages up the context, calls the model over the network (the Messages API), gets the response back, and then runs the tools.

So the accurate equation is:

> what you get when you run the SDK = a local harness (the SDK) + a remote model it phones each turn
>
> The "+" between them is a network request, not a package boundary.

The package you `pip install` is only the **hands**; it needs credentials and a network connection to reach the **brain**.

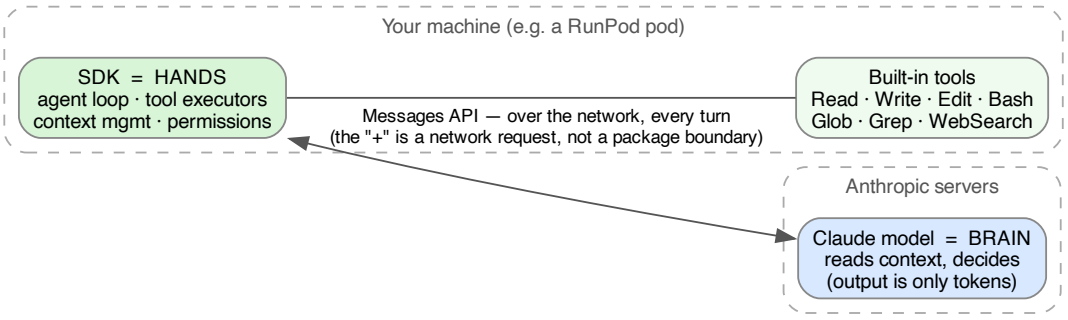

Figure 2: Brain vs hands, and where each lives. The SDK (hands) runs locally with the built-in tools; the model (brain) runs on Anthropic's servers. They talk over the Messages API every turn.

The SDK is the harness — and harness engineering has two levels

That local harness — the loop, the tool executors, context management, permissions, and the built-in tool menu — *is* the harness. This is the bridge into the next chapter: **harness engineering happens**

at two levels.

Level 1 — the harness the Claude SDK gives you out of the box. The agent loop itself, the built-in tools (Read, Write, Edit, Bash, Glob, Grep, WebSearch), context-window management, and the permission system. You get all of this for free just by running the SDK.

Level 2 — the harness engineering you do yourself. On top of the built-in harness, you add your own configuration to give the model more context and structure: the `.claude/` setup — **agents, commands, skills, and `CLAUDE.md`**. This is where you shape *how* the harness behaves for your codebase.

The next chapter, **Harness Engineering**, is entirely about Level 2 — how commands, agents, and skills compose to direct that harness.

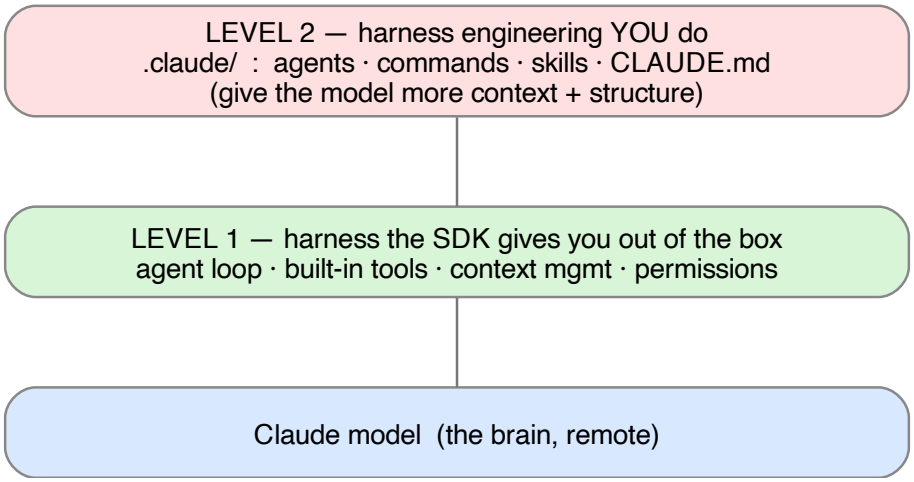

Figure 3: The two levels of harness engineering, stacked on the model. Level 1 comes free with the SDK; Level 2 is the `.claude/` config you author — the subject of the next chapter.

Harness Engineering

> **What this chapter covers:** how Claude Code's *harness* orchestrates work through **commands**, **agents**, and **skills** — who holds the loop, who can spawn whom, and where codebase knowledge lives. Blockquotes are exact quotes from the source session.

Source: a working session on a real backend fleet (`.claude/` of a research-automation repo: orchestrator + be-dev / be-reviewer / be-migration / fe-dev / fe-reviewer). Learned June 2026.

The core distinction: commands vs agents vs skills

Essence: a command is a procedure you run, an agent is a worker you delegate to, a skill is knowledge a worker loads. They don't compete — they stack.

> Command = a procedure you start by typing /name. It runs in your current session — it's a saved playbook injected into the main loop. Agent = a worker you delegate to. It's a separate Claude instance with its own context window, model, and tools. Spawned by the Agent tool, not typed. Skill = knowledge an agent loads to do a job well. Not a worker, not a procedure — reference material pulled into the current context on demand.

	Command	Agent (subagent)	Skill
What it is	a saved prompt / procedure	a delegated worker	on-demand domain knowledge
Lives in	`.claude/commands/*.md`	`.claude/agents/*.md`	`.claude/skills/*/SKILL.md`
Triggered by	human types `/ship`	the Agent tool	the Skill tool
Runs in	your current context	a brand-new isolated context window	enriches the current agent's context
Own model / tools?	no — uses the session's	yes — own model + own tool allowlist	no — not an executor
Returns	nothing — it *is* the session	a summary to whoever spawned it	nothing — just adds knowledge
Who selects it	the human (types `/`)	the orchestrator (reads descriptions)	the model (reads the skill's description)

Analogies to keep:

> Command = the recipe you start… Agent = a line cook you hand a ticket to — a separate person, own station, own tools, own specialty. Comes back with a finished plate (a summary), not their whole process. Skill = the cookbook on the shelf — not a person and not an action, just the reference a cook consults to get the dish right.

a command is the play you call, agents are the players it sends onto the field, and skills are the playbook each player studies before the snap.

How they compose:

You type /ship ← COMMAND (procedure runs in your main session) the orchestrator spawns be-dev ← AGENT (new isolated worker) be-dev loads the backend ← SKILL (reads RLS/decorator/auth_event rules skill before writing into its own context, then codes)

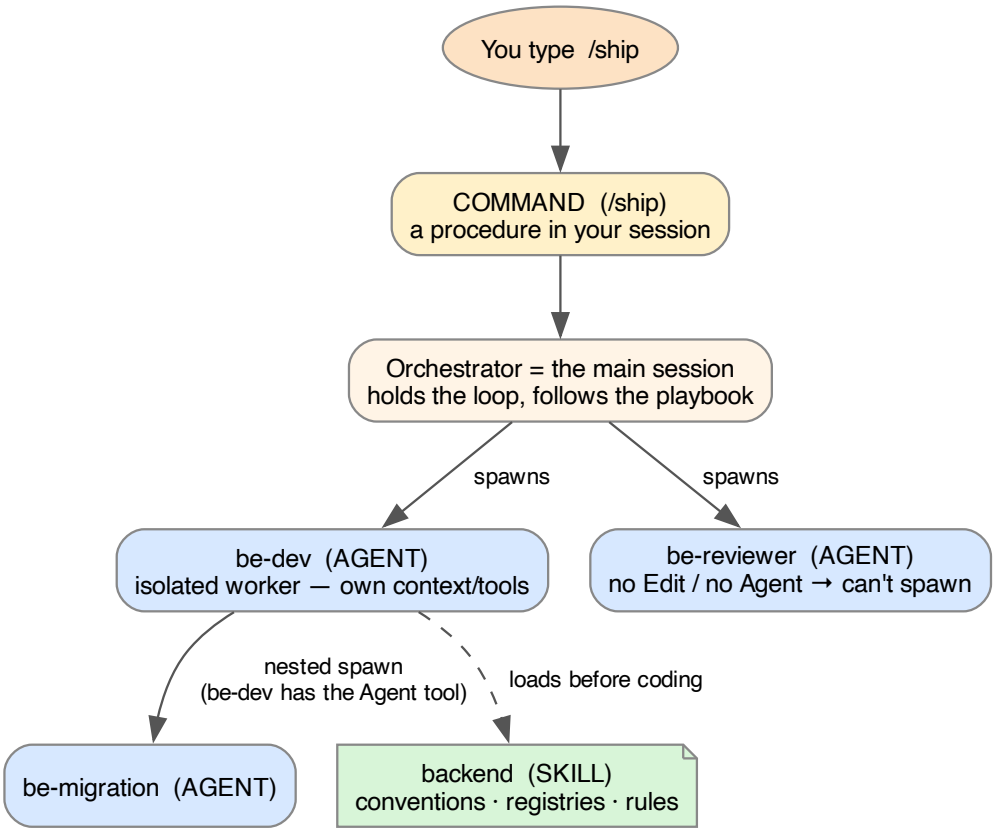

Figure 4: How a command, agents, and a skill compose. The command runs in your session and the orchestrator spawns agents; an agent can nest-spawn only if it has the Agent tool (here be-dev → be-migration); agents load skills for domain knowledge.

Caveat from source:

in recent Claude Code the line between "command" and "user-invocable skill" blurs — some skills can also be launched by typing /name … But the conceptual roles above still hold.

Inside the .md files — what's written differently

The frontmatter keys are the first tell:

- **Agent** is the only one with `tools:` and `model:` — it's an independent executor needing its own capabilities and model. (e.g. `be-dev` has `Edit` + `Agent`; a reviewer wouldn't.)
- **Command** is the only one with `argument-hint:` — it receives `$ARGUMENTS` you type (`/be <path-to-plan>`).
- **Skill** has neither — inert knowledge that borrows the context of whoever loads it.

The body is written in a different voice:

> Agent → "WHO are you and how do you behave?" … a job description + standard operating procedure written in second person to a persona Command → "WHAT sequence do you execute?" … a runbook/algorithm written as imperative control flow: numbered Phases, a Step loop … checkpoints Skill → "WHAT is TRUE about this codebase?" … a reference manual written as declarative facts and tables … literal registries

Crisp summary:

> open an agent file and you read a worker's job description; open a command file and you read a step-by-step procedure; open a skill file and you read a codebase fact-sheet. The tools:/model: keys mark an executor, argument-hint: marks something you invoke with input, and the absence of both marks pure knowledge.

Orchestration, the loop, and who can spawn

The orchestrator holds the loop:

> the main session (the orchestrator) is the thing holding the loop. It decides when to spawn, what type to spawn (be-dev vs be-reviewer vs fe-dev…), and whether it's a subagent (Agent tool) or a Workflow (Workflow tool). The subagents themselves are passive — they get a brief, do the work, return a summary, and disappear. They don't decide when they run.

Refinement 1 — spawning authority follows the tool allowlist:

> Any agent can spawn only if its definition includes the Agent (or Workflow) tool.

In this fleet, the orchestrator spawns all dev/reviewer agents; **be-dev** has the `Agent` tool and spawns **be-migration** itself (a nested spawn); reviewers and fe-dev cannot spawn.

> be-migration is not spun up by the main agent — it's spun up by be-dev (a nested spawn). That's a deliberate design choice: the engineer who hits a schema change is the one who pulls in the migration specialist.

Refinement 2 — the decision is mostly deterministic:

> the "decision" is mostly deterministic, not free-form judgment. The orchestrator follows the playbook in .claude/commands/ship.md … It's executing a procedure, not improvising who to call.

Accurate one-liner:

> the orchestrator decides the top-level spawns by following its command playbook; a subagent can make a nested spawn only if it was granted the Agent tool (here, only be-dev → be-migration).

Who builds the tools

> The tools themselves: yes, built into Claude Code. Agent, Workflow, Read, Edit, Bash, Skill, AskUserQuestion, TodoWrite — these are core capabilities of the Claude Code harness (the CLI/runtime + Agent SDK), not something this repo defined.

Layer	Who built it	Examples
The tools	Claude Code (the harness)	Agent, Workflow, Read, Edit, Bash, Skill
The configuration the tools use	This repo's team (in `.claude/`)	agent defs (`be-dev.md`), skills, slash commands, `settings.json`
External plug-ins	MCP servers registered to the session	`mcp__playwright__*`, Slack MCP

> Claude Code provides the verbs (the tools); this repo's .claude/ folder provides the nouns (which agents, which skills, which permissions); MCP servers add extra verbs from outside.

Two nuances: access is **scoped, not universal** — the harness gives each agent only its `tools:` allowlist (why be-reviewer has no `Edit` or `Agent`); and **MCP tools are the exception** — not built by Claude Code, they come from external servers (Claude Code provides the socket, the tool is third-party).

Where codebase conventions live

Rule: conventions live in the skill, not the agent.

> Skill = the conventions (the codebase facts) … "Hard rules — never break", "Conventions", "Cross-tenant decorators — registry", "Auth-event schema — registry" Agent = the role/behavior, and it points to the skill rather than carrying the conventions itself.

Why:

> Single source of truth. Three different agents — be-dev, be-reviewer, be-migration — all need to know the same RLS rules and decorator registry. If those conventions lived inside each agent file, you'd have three copies to keep in sync.

Conventions actually split across three layers by scope:

Where	What kind of convention	Loaded
`CLAUDE.md` (root + per-dir)	Repo-wide, always-on — naming, commit format, domain language, PR-blocking gates	automatically, every session/agent
Skill (`backend`, `frontend`)	Deep, domain-specific — RLS, decorator registry, auth-event schema, state machines	on demand, when working in that area
Agent (`be-dev…`)	Role-specific behavior only — what this worker does/returns; references the skill	when that agent is spawned

Decision rule:

"What are the rules/invariants of the code?" → skill (deep) or CLAUDE.md (broad, always-on). "What does this particular worker do and how does it behave?" → agent.

Why CLAUDE.md vs skill:

CLAUDE.md is always in context (cheap, broad orientation everyone needs), while a skill is loaded only when relevant (heavier, detailed — you don't want the full RLS/decorator manual in context when you're just writing frontend copy).

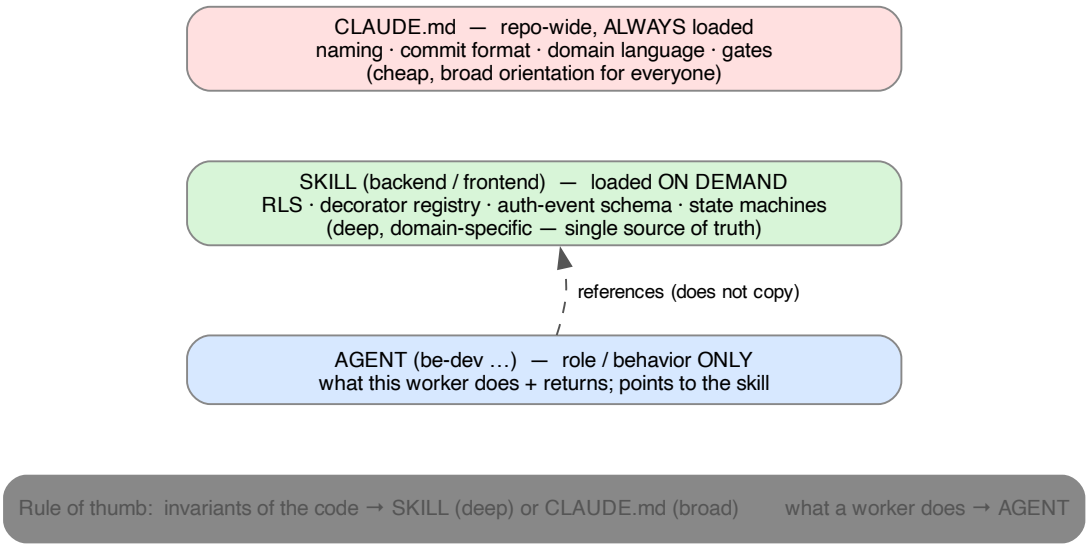

Figure 5: The three layers conventions live in, by scope. CLAUDE.md is broad and always loaded; the skill is deep and loaded on demand; the agent holds only role/behavior and points at the skill.

Deepest "why" lives in ADRs:

Skill = "here's what the practice is, and the working-level why." ADR (docs/adrs/) = "here's the full decision record — the options we weighed and why we chose this." The skill links out to it rather than duplicating it.

The hiring analogy (the whole .claude/ setup)

Piece	Hiring analogy
`CLAUDE.md`	Company-wide onboarding every employee internalizes on day one
Skill (`backend`)	Team engineering wiki — the squad's deep playbook
Agent (`be-dev`)	A specific hire's job description + operating manual
Command (`/ship`)	The tech-lead / sprint process — the manager's SOP for running the team

the command is the manager running the process, the agents are the individual hires it assigns work to, the skill is the team wiki they all consult, and CLAUDE.md is the company handbook everyone already knows.

Note: the agent is a **specific role**, not a generic engineer; and the deepest design rationale sits one layer out in the ADRs the skill links to.

Key takeaways

One-line summary. Command = the coordinator/procedure; agents = the isolated workers it delegates to (and they can nest-spawn if granted the `Agent` tool); skills = the shared domain knowledge any worker (or the main session) loads. Conventions go in skills/CLAUDE.md, never baked into an agent.

Questions to revisit:

- When to author a Workflow vs. plain subagents? (Workflow is only used when you explicitly opt into multi-agent orchestration.)
- How does `settings.json` gate which Bash/MCP calls run without prompting?